Module 10.3 - Reactive Forms: Form Handling

> **Learning Objectives:**
>
> - Monitor form value changes using valueChanges observable
> - Track form status changes using statusChanges observable
> - Update form values using setValue and patchValue
> - Reset forms completely or partially
> - Disable and enable form controls dynamically
> - Handle form submission properly

Table of Contents

---

1. ValueChanges Observable

1.1 Basic ValueChanges

The `valueChanges` observable emits every time a form control's value changes.

Component:

```
import { Component, OnInit } from '@angular/core';
import { FormControl } from '@angular/forms';

@Component({
  selector: 'app-search',
  templateUrl: './search.component.html'
})
export class SearchComponent implements OnInit {
  searchControl = new FormControl('');
  searchHistory: string[] = [];

  ngOnInit() {
    this.searchControl.valueChanges.subscribe(value => {
      console.log('Search value changed:', value);
      if (value && value.length > 2) {
        this.searchHistory.push(value);
      }
    });
  }
}
```

Template:

```
<div>
  <label>Search:</label>
  <input type="text" [formControl]="searchControl">

  <h4>Search History:</h4>
  <ul>
    <li *ngFor="let term of searchHistory">{{ term }}</li>
  </ul>
</div>
```

Output:

User types "a": Console: Search value changed: a User types "an": Console: Search value changed: an User types "ang": Console: Search value changed: ang - "ang" added to search history (length > 2) User types "angu": Console: Search value changed: angu - "angu" added to search history Search History displays: • ang • angu

1.2 ValueChanges on FormGroup

Component:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-calculator',
  templateUrl: './calculator.component.html'
})
export class CalculatorComponent implements OnInit {
  calculatorForm: FormGroup;
  total: number = 0;

  constructor(private fb: FormBuilder) {
    this.calculatorForm = this.fb.group({
      quantity: [0],
      price: [0]
    });
  }

  ngOnInit() {
    this.calculatorForm.valueChanges.subscribe(values => {
      this.total = values.quantity * values.price;
      console.log('Form changed:', values);
      console.log('Total:', this.total);
    });
  }
}
```

Template:

```
<form [formGroup]="calculatorForm">
  <div>
    <label>Quantity:</label>
    <input type="number" formControlName="quantity">
  </div>

  <div>
    <label>Price:</label>
    <input type="number" formControlName="price">
  </div>
```

```
  <h3>Total: {{ total }}</h3>
</form>
```

Output:

```
User enters quantity: 5 Console: Form changed: { quantity: 5, price: 0 } Console:
Total: 0 Display: Total: 0 User enters price: 100 Console: Form changed: { quantity:
5, price: 100 } Console: Total: 500 Display: Total: 500 User changes quantity to 10:
Console: Form changed: { quantity: 10, price: 100 } Console: Total: 1000 Display:
Total: 1000
```

1.3 ValueChanges with Specific Control

Component:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-discount',
  templateUrl: './discount.component.html'
})
export class DiscountComponent implements OnInit {
  orderForm: FormGroup;
  discountMessage: string = '';

  constructor(private fb: FormBuilder) {
    this.orderForm = this.fb.group({
      productName: [''],
      quantity: [1],
      basePrice: [0],
      discount: [0]
    });
  }

  ngOnInit() {
    // Listen to quantity changes only
    this.orderForm.get('quantity')?.valueChanges.subscribe(qty => {
      if (qty >= 10) {
        this.discountMessage = 'Bulk order! 10% discount applied';
        this.orderForm.patchValue({ discount: 10 }, { emitEvent: false });
```

```
      } else if (qty >= 5) {
        this.discountMessage = '5% discount applied';
        this.orderForm.patchValue({ discount: 5 }, { emitEvent: false });
      } else {
        this.discountMessage = 'No discount';
        this.orderForm.patchValue({ discount: 0 }, { emitEvent: false });
      }
    });
  }

  getFinalPrice(): number {
    const base = this.orderForm.value.basePrice * this.orderForm.value.quantity;
    const discount = base * (this.orderForm.value.discount / 100);
    return base - discount;
  }
}
```

Template:

```
<form [formGroup]="orderForm">
  <div>
    <label>Product Name:</label>
    <input type="text" formControlName="productName">
  </div>

  <div>
    <label>Quantity:</label>
    <input type="number" formControlName="quantity">
  </div>

  <div>
    <label>Base Price:</label>
    <input type="number" formControlName="basePrice">
  </div>

  <p>{{ discountMessage }}</p>
  <h3>Final Price: {{ getFinalPrice() }}</h3>
</form>
```

Output:

User enters: - Product Name: Laptop - Quantity: 3 - Base Price: 1000 Display: - No discount - Final Price: 3000 User changes Quantity to 6: - 5% discount applied - Final Price: 5700 (6000 - 5% = 5700) User changes Quantity to 12: - Bulk order! 10% discount applied - Final Price: 10800 (12000 - 10% = 10800)

1.4 Debouncing ValueChanges

Component:

```
import { Component, OnInit } from '@angular/core';
import { FormControl } from '@angular/forms';
import { debounceTime, distinctUntilChanged } from 'rxjs/operators';

@Component({
  selector: 'app-live-search',
  templateUrl: './live-search.component.html'
})
export class LiveSearchComponent implements OnInit {
  searchControl = new FormControl('');
  searchResults: string[] = [];
  searchCount = 0;

  ngOnInit() {
    this.searchControl.valueChanges.pipe(
      debounceTime(500),  // Wait 500ms after user stops typing
      distinctUntilChanged()  // Only emit if value is different
    ).subscribe(searchTerm => {
      this.searchCount++;
      console.log(`Search #${this.searchCount} for:`, searchTerm);
      this.performSearch(searchTerm);
    });
  }

  performSearch(term: string) {
    // Simulate search
    if (term) {
      this.searchResults = [
        `${term} - Result 1`,
        `${term} - Result 2`,
        `${term} - Result 3`
      ];
    } else {
      this.searchResults = [];
```

```
    }
  }
}
```

Template:

```
<div>
  <label>Live Search (with 500ms debounce):</label>
  <input type="text" [formControl]="searchControl">

  <p>Search performed: {{ searchCount }} times</p>

  <ul>
    <li *ngFor="let result of searchResults">{{ result }}</li>
  </ul>
</div>
```

Output:

User types "a" then "n" then "g" quickly (within 500ms): - No search performed yet (waiting for user to stop typing) User stops typing for 500ms: Console: Search #1 for: ang Display: - Search performed: 1 times - ang - Result 1 - ang - Result 2 - ang - Result 3 User continues typing "ular" quickly: User stops: Console: Search #2 for: angular Display: - Search performed: 2 times - angular - Result 1 - angular - Result 2 - angular - Result 3 Note: Without debounce, search would have been performed 7 times (once per keystroke)

---

2. StatusChanges Observable

2.1 Basic StatusChanges

Component:

```
import { Component, OnInit } from '@angular/core';
import { FormControl, Validators } from '@angular/forms';
```

```
@Component({
  selector: 'app-password-strength',
  templateUrl: './password-strength.component.html'
})
export class PasswordStrengthComponent implements OnInit {
  passwordControl = new FormControl('', [
    Validators.required,
    Validators.minLength(6)
  ]);

  statusMessage: string = '';

  ngOnInit() {
    this.passwordControl.statusChanges.subscribe(status => {
      console.log('Form status:', status);

      if (status === 'VALID') {
        this.statusMessage = '✓ Password is valid';
      } else if (status === 'INVALID') {
        this.statusMessage = '✗ Password is invalid';
      } else if (status === 'PENDING') {
        this.statusMessage = '⏳ Validating...';
      }
    });
  }
}
```

Template:

```
<div>
  <label>Password:</label>
  <input type="password" [formControl]="passwordControl">

  <p>{{ statusMessage }}</p>
  <p>Current Status: {{ passwordControl.status }}</p>
</div>
```

Output:

Initial state (empty): Console: Form status: INVALID Display: - ✗ Password is invalid - Current Status: INVALID User types "pass": Console: Form status: INVALID (still less than 6 characters) Display: - ✗ Password is invalid - Current Status: INVALID User

```
types "password": Console: Form status: VALID Display: - ✓ Password is valid - Current Status: VALID User clears the field: Console: Form status: INVALID Display: - ✗ Password is invalid - Current Status: INVALID
```

2.2 StatusChanges on FormGroup

Component:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-registration-status',
  templateUrl: './registration-status.component.html'
})
export class RegistrationStatusComponent implements OnInit {
  registrationForm: FormGroup;
  formStatusHistory: string[] = [];
  canSubmit: boolean = false;

  constructor(private fb: FormBuilder) {
    this.registrationForm = this.fb.group({
      username: ['', [Validators.required, Validators.minLength(3)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  ngOnInit() {
    this.registrationForm.statusChanges.subscribe(status => {
      const timestamp = new Date().toLocaleTimeString();
      this.formStatusHistory.push(`${timestamp}: ${status}`);
      console.log('Form status changed to:', status);

      this.canSubmit = status === 'VALID';
    });
  }
}
```

Template:

```
<form [formGroup]="registrationForm">
  <div>
    <label>Username:</label>
    <input type="text" formControlName="username">
  </div>

  <div>
    <label>Email:</label>
    <input type="email" formControlName="email">
  </div>

  <div>
    <label>Password:</label>
    <input type="password" formControlName="password">
  </div>

  <button [disabled]="!canSubmit">Register</button>

  <h4>Status History:</h4>
  <ul>
    <li *ngFor="let status of formStatusHistory">{{ status }}</li>
  </ul>
</form>
```

Output:

```
Initial: Status History: • (empty) User fills username "jo": Status History: •
14:30:15: INVALID (username too short) User completes username "john": Status History:
• 14:30:15: INVALID • 14:30:17: INVALID (still need email and password) User fills
email "john@test.com": Status History: • 14:30:15: INVALID • 14:30:17: INVALID •
14:30:20: INVALID (still need password) User fills password "secure123": Status
History: • 14:30:15: INVALID • 14:30:17: INVALID • 14:30:20: INVALID • 14:30:23: VALID
Register button enabled
```

3. Setting Form Values

3.1 setValue() - Complete Form Update

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-profile-edit',
  templateUrl: './profile-edit.component.html'
})
export class ProfileEditComponent {
  profileForm: FormGroup;

  savedProfile = {
    name: 'John Doe',
    email: 'john@example.com',
    age: 30
  };

  constructor(private fb: FormBuilder) {
    this.profileForm = this.fb.group({
      name: [''],
      email: [''],
      age: ['']
    });
  }

  loadProfile() {
    // setValue requires ALL form fields
    this.profileForm.setValue(this.savedProfile);
    console.log('Profile loaded with setValue');
  }

  clearProfile() {
    this.profileForm.setValue({
      name: '',
      email: '',
      age: ''
```

```
    });
  }
}
```

Template:

```
<form [formGroup]="profileForm">
  <div>
    <label>Name:</label>
    <input type="text" formControlName="name">
  </div>

  <div>
    <label>Email:</label>
    <input type="email" formControlName="email">
  </div>

  <div>
    <label>Age:</label>
    <input type="number" formControlName="age">
  </div>

  <button type="button" (click)="loadProfile()">Load Profile</button>
  <button type="button" (click)="clearProfile()">Clear</button>
</form>
```

Output:

```
Initial state: All fields empty User clicks "Load Profile": Console: Profile loaded with setValue Display: - Name: John Doe - Email: john@example.com - Age: 30 User clicks "Clear": All fields become empty
```

3.2 patchValue() - Partial Form Update

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
```

```
  selector: 'app-settings',
  templateUrl: './settings.component.html'
})
export class SettingsComponent {
  settingsForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.settingsForm = this.fb.group({
      theme: ['light'],
      language: ['en'],
      notifications: [true],
      autoSave: [false]
    });
  }

  applyLightTheme() {
    // patchValue allows partial updates
    this.settingsForm.patchValue({
      theme: 'light'
    });
    console.log('Light theme applied');
  }

  applyDarkTheme() {
    this.settingsForm.patchValue({
      theme: 'dark'
    });
    console.log('Dark theme applied');
  }

  enableAll() {
    this.settingsForm.patchValue({
      notifications: true,
      autoSave: true
    });
    console.log('All features enabled');
  }
}
```

Template:

```
<form [formGroup]="settingsForm">
  <div>
    <label>Theme: {{ settingsForm.value.theme }}</label>
    <button type="button" (click)="applyLightTheme()">Light</button>
```

```
    <button type="button" (click)="applyDarkTheme()">Dark</button>
  </div>

  <div>
    <label>Language:</label>
    <select formControlName="language">
      <option value="en">English</option>
      <option value="es">Spanish</option>
    </select>
  </div>

  <div>
    <label>
      <input type="checkbox" formControlName="notifications">
      Notifications
    </label>
  </div>

  <div>
    <label>
      <input type="checkbox" formControlName="autoSave">
      Auto Save
    </label>
  </div>

  <button type="button" (click)="enableAll()">Enable All</button>
</form>
```

Output:

Initial state: - Theme: light - Language: en - Notifications: checked - Auto Save: unchecked User clicks "Dark": Console: Dark theme applied - Theme: dark (only theme changed, others unchanged) User unchecks Notifications User clicks "Enable All": Console: All features enabled - Notifications: checked - Auto Save: checked (Language and theme unchanged)

3.3 setValue vs patchValue Comparison

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-value-demo',
  templateUrl: './value-demo.component.html'
})
export class ValueDemoComponent {
  demoForm: FormGroup;
  message: string = '';

  constructor(private fb: FormBuilder) {
    this.demoForm = this.fb.group({
      firstName: [''],
      lastName: [''],
      email: ['']
    });
  }

  trySetValue() {
    try {
      // This will throw ERROR - missing fields
      this.demoForm.setValue({ firstName: 'John' });
      this.message = 'setValue succeeded';
    } catch (error) {
      this.message = 'ERROR: setValue requires all fields!';
      console.error('setValue error:', error);
    }
  }

  tryPatchValue() {
    // This works fine - partial update allowed
    this.demoForm.patchValue({ firstName: 'John' });
    this.message = 'patchValue succeeded (partial update)';
    console.log('patchValue success');
  }

  useSetValueCorrectly() {
    this.demoForm.setValue({
      firstName: 'Jane',
      lastName: 'Smith',
      email: 'jane@example.com'
    });
    this.message = 'setValue succeeded (all fields provided)';
  }
}
```

Template:

```
<form [formGroup]="demoForm">
  <div>
    <label>First Name:</label>
    <input type="text" formControlName="firstName">
  </div>

  <div>
    <label>Last Name:</label>
    <input type="text" formControlName="lastName">
  </div>

  <div>
    <label>Email:</label>
    <input type="email" formControlName="email">
  </div>

  <button type="button" (click)="trySetValue()">
    Try setValue (partial)
  </button>

  <button type="button" (click)="tryPatchValue()">
    Try patchValue (partial)
  </button>

  <button type="button" (click)="useSetValueCorrectly()">
    Use setValue (complete)
  </button>

  <p style="color: red;">{{ message }}</p>
</form>
```

Output:

User clicks "Try setValue (partial)": Console: setValue error: Must supply a value for form control: lastName, email Display: - ERROR: setValue requires all fields! - Form remains empty User clicks "Try patchValue (partial)": Console: patchValue success Display: - patchValue succeeded (partial update) - First Name: John (updated) - Last Name: (empty - not required by patchValue) - Email: (empty) User clicks "Use setValue (complete)": Display: - setValue succeeded (all fields provided) - First Name: Jane - Last Name: Smith - Email: jane@example.com

4. Resetting Forms

4.1 Basic Reset

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-contact-form',
  templateUrl: './contact-form.component.html'
})
export class ContactFormComponent {
  contactForm: FormGroup;
  submitted = false;

  constructor(private fb: FormBuilder) {
    this.contactForm = this.fb.group({
      name: ['', Validators.required],
      email: ['', [Validators.required, Validators.email]],
      message: ['', Validators.required]
    });
  }

  onSubmit() {
    if (this.contactForm.valid) {
      console.log('Form submitted:', this.contactForm.value);
      this.submitted = true;
    }
  }

  onReset() {
    this.contactForm.reset();
    this.submitted = false;
    console.log('Form reset');
  }
}
```

Template:

```
<form [formGroup]="contactForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Name:</label>
    <input type="text" formControlName="name">
  </div>

  <div>
    <label>Email:</label>
    <input type="email" formControlName="email">
  </div>

  <div>
    <label>Message:</label>
    <textarea formControlName="message"></textarea>
  </div>

  <button type="submit">Submit</button>
  <button type="button" (click)="onReset()">Reset</button>

  <p *ngIf="submitted">Thank you! Form submitted successfully.</p>
</form>
```

Output:

User fills form: - Name: Alice - Email: alice@example.com - Message: Hello there! User clicks Submit: Console: Form submitted: { name: "Alice", email: "alice@example.com", message: "Hello there!" } Display: Thank you! Form submitted successfully. User clicks Reset: Console: Form reset - All fields become empty - Message "Thank you!" disappears - Form status reset (pristine, untouched)

4.2 Reset with Default Values

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-preferences',
  templateUrl: './preferences.component.html'
})
```

```
export class PreferencesComponent {
  preferencesForm: FormGroup;

  defaultPreferences = {
    theme: 'light',
    fontSize: 14,
    showSidebar: true
  };

  constructor(private fb: FormBuilder) {
    this.preferencesForm = this.fb.group({
      theme: [this.defaultPreferences.theme],
      fontSize: [this.defaultPreferences.fontSize],
      showSidebar: [this.defaultPreferences.showSidebar]
    });
  }

  resetToDefaults() {
    this.preferencesForm.reset(this.defaultPreferences);
    console.log('Reset to default preferences');
  }

  resetToBlank() {
    this.preferencesForm.reset();
    console.log('Reset to blank (null values)');
  }
}
```

Template:

```
<form [formGroup]="preferencesForm">
  <div>
    <label>Theme:</label>
    <select formControlName="theme">
      <option value="light">Light</option>
      <option value="dark">Dark</option>
    </select>
  </div>

  <div>
    <label>Font Size:</label>
    <input type="number" formControlName="fontSize">
  </div>

  <div>
```

```
    <label>
      <input type="checkbox" formControlName="showSidebar">
      Show Sidebar
    </label>
  </div>

  <button type="button" (click)="resetToDefaults()">
    Reset to Defaults
  </button>

  <button type="button" (click)="resetToBlank()">
    Reset to Blank
  </button>
</form>
```

Output:

```
Initial state (defaults): - Theme: light - Font Size: 14 - Show Sidebar: checked User changes: - Theme: dark - Font Size: 18 - Show Sidebar: unchecked User clicks "Reset to Defaults": Console: Reset to default preferences Display: - Theme: light (back to default) - Font Size: 14 (back to default) - Show Sidebar: checked (back to default) User changes again: - Theme: dark - Font Size: 20 User clicks "Reset to Blank": Console: Reset to blank (null values) Display: - Theme: (null/blank) - Font Size: (null/blank) - Show Sidebar: unchecked (checkboxes default to false)
```

4.3 Partial Reset

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-filter-form',
  templateUrl: './filter-form.component.html'
})
export class FilterFormComponent {
  filterForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.filterForm = this.fb.group({
```

```
      category: [''],
      priceMin: [''],
      priceMax: [''],
      inStock: [false]
    });
  }

  resetPriceOnly() {
    // Reset only price fields
    this.filterForm.patchValue({
      priceMin: '',
      priceMax: ''
    });
    console.log('Price filters reset');
  }

  resetAll() {
    this.filterForm.reset();
    console.log('All filters reset');
  }
}
```

Template:

```
<form [formGroup]="filterForm">
  <div>
    <label>Category:</label>
    <select formControlName="category">
      <option value="">All</option>
      <option value="electronics">Electronics</option>
      <option value="books">Books</option>
    </select>
  </div>

  <div>
    <label>Min Price:</label>
    <input type="number" formControlName="priceMin">
  </div>

  <div>
    <label>Max Price:</label>
    <input type="number" formControlName="priceMax">
  </div>

  <div>
```

```
    <label>
      <input type="checkbox" formControlName="inStock">
      In Stock Only
    </label>
  </div>

  <button type="button" (click)="resetPriceOnly()">
    Reset Price
  </button>

  <button type="button" (click)="resetAll()">
    Reset All
  </button>
</form>
```

Output:

```
User sets filters: - Category: electronics - Min Price: 100 - Max Price: 500 - In Stock: checked User clicks "Reset Price": Console: Price filters reset Display: - Category: electronics (unchanged) - Min Price: (empty) - Max Price: (empty) - In Stock: checked (unchanged) User clicks "Reset All": Console: All filters reset Display: - Category: (empty) - Min Price: (empty) - Max Price: (empty) - In Stock: unchecked
```

---

5. Disabling and Enabling Controls

5.1 Disable/Enable Single Control

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-shipping-form',
  templateUrl: './shipping-form.component.html'
})
export class ShippingFormComponent {
  shippingForm: FormGroup;
```

```
  constructor(private fb: FormBuilder) {
    this.shippingForm = this.fb.group({
      sameAsbilling: [false],
      shippingAddress: ['']
    });
  }

  onSameAsBillingChange(checked: boolean) {
    if (checked) {
      this.shippingForm.get('shippingAddress')?.disable();
      console.log('Shipping address disabled');
    } else {
      this.shippingForm.get('shippingAddress')?.enable();
      console.log('Shipping address enabled');
    }
  }
}
```

Template:

```
<form [formGroup]="shippingForm">
  <div>
    <label>
      <input type="checkbox"
             formControlName="sameAsbilling"
             (change)="onSameAsBillingChange($any($event.target).checked)">
      Same as Billing Address
    </label>
  </div>

  <div>
    <label>Shipping Address:</label>
    <input type="text" formControlName="shippingAddress">
  </div>

  <p>Address Enabled: {{ shippingForm.get('shippingAddress')?.enabled }}</p>
  <p>Address Disabled: {{ shippingForm.get('shippingAddress')?.disabled }}</p>
</form>
```

Output:

> Initial state: - Same as Billing: unchecked - Shipping Address: enabled (user can type) - Address Enabled: true - Address Disabled: false User checks "Same as Billing Address": Console: Shipping address disabled Display: - Shipping Address: disabled (greyed out, cannot type) - Address Enabled: false - Address Disabled: true User unchecks the box: Console: Shipping address enabled Display: - Shipping Address: enabled (can type again) - Address Enabled: true - Address Disabled: false

5.2 Disable/Enable FormGroup

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-payment-form',
  templateUrl: './payment-form.component.html'
})
export class PaymentFormComponent {
  paymentForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.paymentForm = this.fb.group({
      paymentMethod: ['cash'],
      cardDetails: this.fb.group({
        cardNumber: [''],
        expiryDate: [''],
        cvv: ['']
      })
    });

    // Initially disable card details
    this.paymentForm.get('cardDetails')?.disable();
  }

  onPaymentMethodChange(method: string) {
    if (method === 'card') {
      this.paymentForm.get('cardDetails')?.enable();
      console.log('Card details enabled');
    } else {
      this.paymentForm.get('cardDetails')?.disable();
      console.log('Card details disabled');
```

```
    }
  }
}
```

Template:

```
<form [formGroup]="paymentForm">
  <div>
    <label>Payment Method:</label>
    <select formControlName="paymentMethod"
            (change)="onPaymentMethodChange($any($event.target).value)">
      <option value="cash">Cash</option>
      <option value="card">Credit Card</option>
    </select>
  </div>

  <div formGroupName="cardDetails">
    <h4>Card Details</h4>

    <div>
      <label>Card Number:</label>
      <input type="text" formControlName="cardNumber">
    </div>

    <div>
      <label>Expiry Date:</label>
      <input type="text" formControlName="expiryDate">
    </div>

    <div>
      <label>CVV:</label>
      <input type="text" formControlName="cvv">
    </div>
  </div>
</form>
```

Output:

Initial state: - Payment Method: cash - All card detail fields: disabled (greyed out) User selects "Credit Card": Console: Card details enabled Display: - All card fields become enabled - User can enter: • Card Number: 4111111111111111 • Expiry Date: 12/25 • CVV: 123 User selects "Cash": Console: Card details disabled Display: - All card fields become disabled again - Entered values remain but cannot be edited

5.3 Getting Values from Disabled Controls

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-user-form',
  templateUrl: './user-form.component.html'
})
export class UserFormComponent {
  userForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.userForm = this.fb.group({
      userId: [{ value: 'USR001', disabled: true }],
      username: [''],
      email: ['']
    });
  }

  getFormValue() {
    // Does NOT include disabled fields
    console.log('form.value:', this.userForm.value);
  }

  getFormRawValue() {
    // Includes disabled fields
    console.log('form.getRawValue():', this.userForm.getRawValue());
  }
}
```

Template:

```
<form [formGroup]="userForm">
  <div>
    <label>User ID:</label>
    <input type="text" formControlName="userId">
  </div>

  <div>
    <label>Username:</label>
    <input type="text" formControlName="username">
  </div>

  <div>
    <label>Email:</label>
    <input type="email" formControlName="email">
  </div>

  <button type="button" (click)="getFormValue()">
    Get form.value
  </button>

  <button type="button" (click)="getFormRawValue()">
    Get form.getRawValue()
  </button>
</form>
```

Output:

User fills form: - User ID: USR001 (disabled, greyed out) - Username: john_doe - Email: john@example.com User clicks "Get form.value": Console: form.value: { username: "john_doe", email: "john@example.com" } Note: userId is NOT included (it's disabled) User clicks "Get form.getRawValue()": Console: form.getRawValue(): { userId: "USR001", username: "john_doe", email: "john@example.com" } Note: userId IS included (even though disabled)

6. Form Submission Handling

6.1 Basic Form Submission

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-feedback-form',
  templateUrl: './feedback-form.component.html'
})
export class FeedbackFormComponent {
  feedbackForm: FormGroup;
  submitted = false;
  submittedData: any = null;

  constructor(private fb: FormBuilder) {
    this.feedbackForm = this.fb.group({
      name: ['', Validators.required],
      rating: ['', Validators.required],
      comments: ['']
    });
  }

  onSubmit() {
    this.submitted = true;

    if (this.feedbackForm.valid) {
      this.submittedData = this.feedbackForm.value;
      console.log('Feedback submitted:', this.submittedData);

      // Simulate API call
      setTimeout(() => {
        alert('Thank you for your feedback!');
        this.feedbackForm.reset();
        this.submitted = false;
      }, 1000);
    } else {
      console.log('Form is invalid');
    }
  }
```

```
  get name() {
    return this.feedbackForm.get('name');
  }

  get rating() {
    return this.feedbackForm.get('rating');
  }
}
```

Template:

```
<form [formGroup]="feedbackForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Name:</label>
    <input type="text" formControlName="name">
    <div *ngIf="name?.invalid && (name?.touched || submitted)" style="color: red;">
      Name is required
    </div>
  </div>

  <div>
    <label>Rating:</label>
    <select formControlName="rating">
      <option value="">Select Rating</option>
      <option value="5">5 - Excellent</option>
      <option value="4">4 - Good</option>
      <option value="3">3 - Average</option>
      <option value="2">2 - Poor</option>
      <option value="1">1 - Very Poor</option>
    </select>
    <div *ngIf="rating?.invalid && (rating?.touched || submitted)" style="color: red;">
      Rating is required
    </div>
  </div>

  <div>
    <label>Comments:</label>
    <textarea formControlName="comments"></textarea>
  </div>

  <button type="submit">Submit Feedback</button>
</form>
```

Output:

User clicks "Submit Feedback" without filling: - Error: "Name is required" appears - Error: "Rating is required" appears Console: Form is invalid User fills: - Name: Sarah - Rating: 5 - Excellent - Comments: Great service! User clicks Submit: Console: Feedback submitted: { name: "Sarah", rating: "5", comments: "Great service!" } After 1 second: - Alert: "Thank you for your feedback!" - Form resets - Errors disappear

6.2 Preventing Multiple Submissions

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-order-form',
  templateUrl: './order-form.component.html'
})
export class OrderFormComponent {
  orderForm: FormGroup;
  isSubmitting = false;

  constructor(private fb: FormBuilder) {
    this.orderForm = this.fb.group({
      product: ['', Validators.required],
      quantity: [1, [Validators.required, Validators.min(1)]]
    });
  }

  onSubmit() {
    if (this.orderForm.valid && !this.isSubmitting) {
      this.isSubmitting = true;
      this.orderForm.disable();

      console.log('Processing order:', this.orderForm.getRawValue());

      // Simulate API call (3 seconds)
      setTimeout(() => {
        console.log('Order placed successfully');
        this.isSubmitting = false;
        this.orderForm.enable();
```

```
        this.orderForm.reset({ quantity: 1 });
      }, 3000);
    }
  }
}
```

Template:

```
<form [formGroup]="orderForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Product:</label>
    <input type="text" formControlName="product">
  </div>

  <div>
    <label>Quantity:</label>
    <input type="number" formControlName="quantity">
  </div>

  <button type="submit" [disabled]="orderForm.invalid || isSubmitting">
    {{ isSubmitting ? 'Processing...' : 'Place Order' }}
  </button>

  <p *ngIf="isSubmitting">Please wait while we process your order...</p>
</form>
```

Output:

User fills: - Product: Laptop - Quantity: 2 User clicks "Place Order": Console: Processing order: { product: "Laptop", quantity: 2 } Display: - Button text changes to "Processing..." - Button becomes disabled - All form fields disabled - Message: "Please wait while we process your order..." User tries to click button again (nothing happens - disabled) After 3 seconds: Console: Order placed successfully Display: - Button text: "Place Order" - Button enabled - Form fields enabled - Form reset with quantity: 1

7. Practical Example: Product Order Management

Let's build a complete product order system using all form handling techniques.

Service (product.service.ts):

```
import { Injectable } from '@angular/core';

export interface Product {
  id: string;
  name: string;
  price: number;
  stock: number;
}

@Injectable({
  providedIn: 'root'
})
export class ProductService {
  products: Product[] = [
    { id: 'P001', name: 'Laptop', price: 1200, stock: 5 },
    { id: 'P002', name: 'Mouse', price: 25, stock: 50 },
    { id: 'P003', name: 'Keyboard', price: 75, stock: 30 },
    { id: 'P004', name: 'Monitor', price: 300, stock: 10 }
  ];

  getProduct(id: string): Product | undefined {
    return this.products.find(p => p.id === id);
  }

  submitOrder(orderData: any): Promise {
    return new Promise((resolve) => {
      setTimeout(() => {
        console.log('Order submitted to server:', orderData);
        resolve(true);
      }, 2000);
    });
  }
}
```

Component (order-management.component.ts):

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { FormBuilder, FormGroup, FormArray, Validators } from '@angular/forms';
import { Subscription } from 'rxjs';
import { debounceTime } from 'rxjs/operators';
import { ProductService, Product } from './product.service';

@Component({
  selector: 'app-order-management',
  templateUrl: './order-management.component.html',
  styles: [`
    .container {
      max-width: 800px;
      margin: 20px auto;
      padding: 20px;
    }
    .form-section {
      background: #f9f9f9;
      padding: 20px;
      margin: 15px 0;
      border-radius: 8px;
      border: 1px solid #ddd;
    }
    .form-section h3 {
      margin-top: 0;
      color: #007bff;
    }
    .form-group {
      margin: 15px 0;
    }
    label {
      display: block;
      font-weight: bold;
      margin-bottom: 5px;
    }
    input, select {
      width: 100%;
      padding: 8px;
      border: 1px solid #ddd;
      border-radius: 4px;
      box-sizing: border-box;
    }
    input:disabled, select:disabled {
      background: #e9ecef;
      cursor: not-allowed;
    }
    .order-item {
```

```
    background: white;
    padding: 15px;
    margin: 10px 0;
    border-radius: 5px;
    border: 1px solid #ddd;
  }
  .order-summary {
    background: #e7f3ff;
    padding: 15px;
    border-radius: 5px;
    margin: 15px 0;
  }
  .summary-row {
    display: flex;
    justify-content: space-between;
    margin: 5px 0;
  }
  .summary-row.total {
    font-size: 1.2em;
    font-weight: bold;
    border-top: 2px solid #007bff;
    padding-top: 10px;
    margin-top: 10px;
  }
  button {
    padding: 10px 20px;
    margin: 5px 5px 5px 0;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
  }
  .btn-primary {
    background: #007bff;
    color: white;
  }
  .btn-primary:disabled {
    background: #ccc;
    cursor: not-allowed;
  }
  .btn-danger {
    background: #dc3545;
    color: white;
  }
  .btn-success {
    background: #28a745;
    color: white;
```

```
    }
    .error {
      color: #dc3545;
      font-size: 12px;
      margin-top: 3px;
    }
    .success-message {
      background: #d4edda;
      color: #155724;
      padding: 20px;
      border-radius: 5px;
      text-align: center;
      margin: 20px 0;
    }
    .status-info {
      background: #fff3cd;
      padding: 10px;
      border-radius: 4px;
      margin: 10px 0;
      font-size: 13px;
    }
  `]
})
export class OrderManagementComponent implements OnInit, OnDestroy {
  orderForm: FormGroup;
  isSubmitting = false;
  orderSubmitted = false;
  submittedOrderData: any = null;

  totalAmount = 0;
  totalItems = 0;

  private subscriptions: Subscription[] = [];

  constructor(
    private fb: FormBuilder,
    public productService: ProductService
  ) {
    this.orderForm = this.fb.group({
      customer: this.fb.group({
        name: ['', Validators.required],
        email: ['', [Validators.required, Validators.email]],
        phone: ['', Validators.required]
      }),
      shipping: this.fb.group({
        expressShipping: [false],
        address: ['', Validators.required]
```

```
    }),
    orderItems: this.fb.array([]),
    specialInstructions: ['']
  });
}

ngOnInit() {
  // Monitor express shipping changes
  const shippingSub = this.orderForm.get('shipping.expressShipping')?.valueChanges
    .subscribe(isExpress => {
      console.log('Express shipping:', isExpress);
      this.calculateTotal();
    });

  if (shippingSub) {
    this.subscriptions.push(shippingSub);
  }

  // Monitor form status changes
  const statusSub = this.orderForm.statusChanges.subscribe(status => {
    console.log('Form status:', status);
  });

  if (statusSub) {
    this.subscriptions.push(statusSub);
  }

  // Add first order item
  this.addOrderItem();
}

ngOnDestroy() {
  this.subscriptions.forEach(sub => sub.unsubscribe());
}

get customer() {
  return this.orderForm.get('customer') as FormGroup;
}

get shipping() {
  return this.orderForm.get('shipping') as FormGroup;
}

get orderItems() {
  return this.orderForm.get('orderItems') as FormArray;
}
```

```
  addOrderItem() {
    const orderItem = this.fb.group({
      product: ['', Validators.required],
      quantity: [1, [Validators.required, Validators.min(1)]],
      unitPrice: [{ value: 0, disabled: true }],
      subtotal: [{ value: 0, disabled: true }]
    });

    // Listen to product selection
    const productSub = orderItem.get('product')?.valueChanges.subscribe(productId => {
      const product = this.productService.getProduct(productId);
      if (product) {
        orderItem.patchValue({
          unitPrice: product.price,
          quantity: 1
        }, { emitEvent: false });
        this.updateSubtotal(orderItem);
      }
    });

    // Listen to quantity changes
    const qtySub = orderItem.get('quantity')?.valueChanges
      .pipe(debounceTime(300))
      .subscribe(() => {
        this.updateSubtotal(orderItem);
      });

    if (productSub) this.subscriptions.push(productSub);
    if (qtySub) this.subscriptions.push(qtySub);

    this.orderItems.push(orderItem);
  }

  removeOrderItem(index: number) {
    this.orderItems.removeAt(index);
    this.calculateTotal();
  }

  updateSubtotal(itemGroup: FormGroup) {
    const quantity = itemGroup.get('quantity')?.value || 0;
    const unitPrice = itemGroup.get('unitPrice')?.value || 0;
    const subtotal = quantity * unitPrice;

    itemGroup.patchValue({ subtotal }, { emitEvent: false });
    this.calculateTotal();
  }
```

```
  calculateTotal() {
    let total = 0;
    let items = 0;

    this.orderItems.controls.forEach(control => {
      const itemGroup = control as FormGroup;
      total += itemGroup.get('subtotal')?.value || 0;
      items += itemGroup.get('quantity')?.value || 0;
    });

    // Add express shipping cost
    if (this.shipping.get('expressShipping')?.value) {
      total += 50;
    }

    this.totalAmount = total;
    this.totalItems = items;
  }

  loadSampleData() {
    this.orderForm.patchValue({
      customer: {
        name: 'John Doe',
        email: 'john@example.com',
        phone: '5551234567'
      },
      shipping: {
        address: '123 Main St, City, State 12345'
      }
    });

    // Clear existing items
    while (this.orderItems.length > 0) {
      this.orderItems.removeAt(0);
    }

    // Add sample items
    this.addOrderItem();
    this.orderItems.at(0).patchValue({
      product: 'P001',
      quantity: 2
    });

    console.log('Sample data loaded');
  }

  async onSubmit() {
```

```
  if (this.orderForm.valid && !this.isSubmitting) {
    this.isSubmitting = true;
    this.orderForm.disable();

    // Get raw value to include disabled fields
    const orderData = {
      ...this.orderForm.getRawValue(),
      totalAmount: this.totalAmount,
      totalItems: this.totalItems,
      orderDate: new Date()
    };

    try {
      const success = await this.productService.submitOrder(orderData);

      if (success) {
        this.submittedOrderData = orderData;
        this.orderSubmitted = true;
        console.log('Order placed successfully');
      }
    } catch (error) {
      console.error('Order submission failed:', error);
    } finally {
      this.isSubmitting = false;
      this.orderForm.enable();
    }
  } else {
    console.log('Form is invalid or already submitting');
    this.markFormGroupTouched(this.orderForm);
  }
}

markFormGroupTouched(formGroup: FormGroup | FormArray) {
  Object.keys(formGroup.controls).forEach(key => {
    const control = formGroup.get(key);
    control?.markAsTouched();

    if (control instanceof FormGroup || control instanceof FormArray) {
      this.markFormGroupTouched(control);
    }
  });
}

resetForm() {
  this.orderForm.reset({
    shipping: { expressShipping: false }
  });
```

```
    // Clear order items
    while (this.orderItems.length > 0) {
      this.orderItems.removeAt(0);
    }

    // Add first item
    this.addOrderItem();

    this.totalAmount = 0;
    this.totalItems = 0;
    console.log('Form reset');
  }

  startNewOrder() {
    this.orderSubmitted = false;
    this.resetForm();
  }
}
```

Template (order-management.component.html):

```
<div class="container">
  <h2>Product Order Management</h2>

  <!-- Success Message -->
  <div *ngIf="orderSubmitted" class="success-message">
    <h3>✓ Order Placed Successfully!</h3>
    <p><strong>Order Date:</strong> {{ submittedOrderData?.orderDate | date:'medium' }}</p>
    <p><strong>Customer:</strong> {{ submittedOrderData?.customer.name }}</p>
    <p><strong>Total Items:</strong> {{ submittedOrderData?.totalItems }}</p>
    <p><strong>Total Amount:</strong> ${{ submittedOrderData?.totalAmount }}</p>
    <button class="btn-primary" (click)="startNewOrder()">New Order</button>
  </div>

  <!-- Order Form -->
  <form [formGroup]="orderForm" (ngSubmit)="onSubmit()" *ngIf="!orderSubmitted">

    <!-- Customer Information -->
    <div class="form-section" formGroupName="customer">
      <h3>Customer Information</h3>

      <div class="form-group">
        <label>Name *</label>
        <input type="text" formControlName="name">
```

```
      <div *ngIf="customer.get('name')?.invalid && customer.get('name')?.touched"
           class="error">
        Name is required
      </div>
    </div>

    <div class="form-group">
      <label>Email *</label>
      <input type="email" formControlName="email">
      <div *ngIf="customer.get('email')?.invalid && customer.get('email')?.touched"
           class="error">
        Valid email is required
      </div>
    </div>

    <div class="form-group">
      <label>Phone *</label>
      <input type="tel" formControlName="phone">
      <div *ngIf="customer.get('phone')?.invalid && customer.get('phone')?.touched"
           class="error">
        Phone is required
      </div>
    </div>
  </div>

  <!-- Shipping Information -->
  <div class="form-section" formGroupName="shipping">
    <h3>Shipping Information</h3>

    <div class="form-group">
      <label>
        <input type="checkbox" formControlName="expressShipping">
        Express Shipping (+$50)
      </label>
    </div>

    <div class="form-group">
      <label>Shipping Address *</label>
      <input type="text" formControlName="address">
      <div *ngIf="shipping.get('address')?.invalid && shipping.get('address')?.touched"
           class="error">
        Shipping address is required
      </div>
    </div>
  </div>

  <!-- Order Items -->
```

```
    <div class="form-section">
      <h3>Order Items</h3>

      <div formArrayName="orderItems">
        <div *ngFor="let item of orderItems.controls; let i = index"
             [formGroupName]="i"
             class="order-item">
          <h4>Item {{ i + 1 }}</h4>

          <div class="form-group">
            <label>Product *</label>
            <select formControlName="product">
              <option value="">Select Product</option>
              <option *ngFor="let product of productService.products" [value]="product.id">
                {{ product.name }} - ${{ product.price }} (Stock: {{ product.stock }})
              </option>
            </select>
          </div>

          <div class="form-group">
            <label>Quantity *</label>
            <input type="number" formControlName="quantity">
          </div>

          <div class="form-group">
            <label>Unit Price</label>
            <input type="text" formControlName="unitPrice">
          </div>

          <div class="form-group">
            <label>Subtotal</label>
            <input type="text" formControlName="subtotal">
          </div>

          <button type="button"
                  class="btn-danger"
                  (click)="removeOrderItem(i)"
                  *ngIf="orderItems.length > 1">
            Remove Item
          </button>
        </div>

        <button type="button"
                class="btn-primary"
                (click)="addOrderItem()">
          Add Another Item
        </button>
```

```
  </div>
</div>

<!-- Special Instructions -->
<div class="form-section">
  <h3>Special Instructions (Optional)</h3>
  <textarea formControlName="specialInstructions"
            rows="3"
            style="width: 100%;"></textarea>
</div>

<!-- Order Summary -->
<div class="order-summary">
  <h3>Order Summary</h3>
  <div class="summary-row">
    <span>Total Items:</span>
    <span>{{ totalItems }}</span>
  </div>
  <div class="summary-row">
    <span>Subtotal:</span>
    <span>${{ totalAmount - (shipping.value.expressShipping ? 50 : 0) }}</span>
  </div>
  <div class="summary-row" *ngIf="shipping.value.expressShipping">
    <span>Express Shipping:</span>
    <span>$50</span>
  </div>
  <div class="summary-row total">
    <span>Total Amount:</span>
    <span>${{ totalAmount }}</span>
  </div>
</div>

<!-- Form Actions -->
<div>
  <button type="submit"
          class="btn-success"
          [disabled]="orderForm.invalid || isSubmitting">
    {{ isSubmitting ? 'Processing Order...' : 'Place Order' }}
  </button>

  <button type="button"
          class="btn-primary"
          (click)="loadSampleData()"
          [disabled]="isSubmitting">
    Load Sample Data
  </button>
```

```
      <button type="button"
              (click)="resetForm()"
              [disabled]="isSubmitting">
        Reset Form
      </button>
    </div>

    <!-- Form Status -->
    <div class="status-info">
      <strong>Form Status:</strong>
      Valid: {{ orderForm.valid }} |
      Dirty: {{ orderForm.dirty }} |
      Touched: {{ orderForm.touched }}
    </div>

    <div *ngIf="isSubmitting" class="status-info">
      ⏳ Processing your order, please wait...
    </div>
  </form>
</div>
```

Output:

User clicks "Load Sample Data": Console: Sample data loaded Form populates with: - Customer: John Doe, john@example.com, 5551234567 - Shipping: 123 Main St, City, State 12345 - Item 1: Laptop (2 units, $1200 each, subtotal $2400) User clicks "Add Another Item": - Item 2 appears User selects: - Product: Mouse - Quantity: 5 Real-time updates: - Unit Price: $25 (auto-filled) - Subtotal: $125 (auto-calculated after 300ms debounce) Order Summary updates: - Total Items: 7 (2 + 5) - Subtotal: $2525 - Total Amount: $2525 User checks "Express Shipping": Console: Express shipping: true Order Summary updates: - Express Shipping: $50 - Total Amount: $2575 User clicks "Place Order": Form Status: Valid: true Console: Form status: VALID - All fields disabled - Button text: "Processing Order..." - Status: "⏳ Processing your order, please wait..." After 2 seconds: Console: Order submitted to server: { customer: {...}, shipping: {...}, ... } Console: Order placed successfully Success message displays: ✓ Order Placed Successfully! - Order Date: Feb 2, 2026, 2:30:45 PM - Customer: John Doe - Total Items: 7 - Total Amount: $2575 User clicks "New Order": - Form resets - Ready for new order

8. Practical Task

Task: Build an Event Registration System with Complete Form Handling

Requirements:

1. **Create an Event Registration Form with:**

   Participant Information (FormGroup):

   - Full Name (required)
   - Email (required, email validation)
   - Phone (required, pattern: 10 digits)
   - Organization (optional)

   Event Selection (FormGroup):

   - Event Type (dropdown: Conference, Workshop, Seminar, Webinar) (required)
   - Event Date (date picker) (required)
   - Number of Attendees (number, min 1, max 10) (required)
   - VIP Pass (checkbox)

   Sessions (FormArray):

   - Each session has:
     - Session Name (required)
     - Duration (dropdown: 1 hour, 2 hours, 3 hours) (required)
     - Cost per session (disabled, auto-calculated based on duration and VIP)
   - Cost calculation:
     - Regular: 1 hour = $50, 2 hours = $90, 3 hours = $120
     - VIP (20% discount): 1 hour = $40, 2 hours = $72, 3 hours = $96

   Payment Summary (Auto-calculated):

   - Total Sessions
   - Base Amount

- VIP Discount (if applicable)
- Total Amount

2. **Implement Form Handling:**

   - **ValueChanges:**
     - Monitor VIP Pass changes → recalculate all session costs
     - Monitor session duration changes → update individual session cost
     - Use debounceTime(300) for number of attendees

   - **StatusChanges:**
     - Track form validity status
     - Display real-time validation status

   - **Set/Patch Values:**
     - "Load Demo Data" button to populate form
     - Partial updates for VIP calculations

   - **Reset:**
     - Reset button with default values (1 attendee, VIP unchecked)
     - Clear sessions array

   - **Disable/Enable:**
     - Disable entire form during submission
     - Session cost fields always disabled (calculated)

   - **Submission:**
     - Prevent multiple submissions
     - Show loading state
     - Simulate 2-second API call
     - Show success message with registration details
     - Use getRawValue() to include disabled fields

3. **Create Service:**

```
EventService with:
- getSessionCost(duration: string, isVIP: boolean): number
- submitRegistration(data: any): Promise<boolean>
```

4. **Display Requirements:**
    - Real-time cost calculation as sessions are added/modified
    - Show validation errors only when touched
    - Display form status (valid/invalid, dirty, touched)
    - Show submission progress
    - Success message with all registration details

Expected Console Output:

```
VIP Pass changed: true
Session cost recalculated for all sessions

Registration submitted: {
  participant: {
    fullName: "Priya Sharma",
    email: "priya@company.com",
    phone: "9876543210",
    organization: "Tech Corp"
  },
  event: {
    eventType: "Conference",
    eventDate: "2026-03-15",
    numberOfAttendees: 3,
    vipPass: true
  },
  sessions: [
    { sessionName: "AI Fundamentals", duration: "2 hours", cost: 72 },
    { sessionName: "Cloud Computing", duration: "3 hours", cost: 96 }
  ],
  totalSessions: 2,
  baseAmount: 210,
  vipDiscount: 42,
  totalAmount: 168
}
Registration confirmed successfully!
```

9. Summary

Key Concepts Covered

Concept	Description	Use Case
valueChanges	Observable that emits when form value changes	Real-time calculations, search
statusChanges	Observable that emits when form status changes	Form validity tracking
setValue()	Set complete form value (all fields required)	Loading saved data
patchValue()	Set partial form value (flexible)	Partial updates
reset()	Reset form to initial state	Clear form after submission
disable()/enable()	Control form interactivity	Conditional fields, submission state
getRawValue()	Get values including disabled controls	Submission with disabled fields

Observable Operators for Forms

Operator	Purpose	Example Use
`debounceTime(ms)`	Wait specified time before emitting	Search, expensive calculations
`distinctUntilChanged()`	Only emit if value changed	Prevent duplicate API calls
`map()`	Transform emitted values	Format data before using
`filter()`	Only emit if condition true	Validate before processing

Best Practices

1. **ValueChanges Usage:**
   - Always unsubscribe in ngOnDestroy()
   - Use debounceTime for expensive operations
   - Use { emitEvent: false } to prevent infinite loops
2. **Setting Values:**
   - Use setValue() when you have complete data
   - Use patchValue() for partial updates
   - Remember setValue() throws error if fields missing
3. **Resetting Forms:**
   - Pass default values to reset() when needed
   - Reset validation states with form reset
   - Clear FormArrays manually if needed
4. **Disabling Controls:**
   - Use getRawValue() to include disabled fields
   - Disable entire form during submission
   - Remember to re-enable after operation
5. **Form Submission:**
   - Prevent multiple submissions with flag
   - Disable form during submission
   - Mark all fields as touched to show errors
   - Handle errors gracefully

Common Patterns

ValueChanges with Debounce:

```
control.valueChanges.pipe(
  debounceTime(500),
  distinctUntilChanged()
```

```
).subscribe(value => {
  // Handle value
});
```

Prevent Infinite Loop:

```
control.patchValue(newValue, { emitEvent: false });
```

Mark All Touched:

```
Object.keys(form.controls).forEach(key => {
  form.get(key)?.markAsTouched();
});
```

Safe Submission:

```
if (form.valid && !isSubmitting) {
  isSubmitting = true;
  form.disable();
  // Submit logic
  // Finally: isSubmitting = false; form.enable();
}
```

Form Value Methods Comparison

Method	Includes Disabled?	When to Use
`form.value`	No	Display, validation checks
`form.getRawValue()`	Yes	Submission, saving complete data

What's Next?

You've now completed the Reactive Forms modules! Next topics to explore:

- **HTTP Client:** Making API calls to backend

- **Routing:** Navigation between components
- **State Management:** Managing application state
- **Advanced Patterns:** Dynamic forms, custom form controls

> **Module Complete!** You now have comprehensive knowledge of reactive forms handling including value monitoring, form updates, resets, enabling/disabling controls, and proper form submission. You can build production-ready applications with complete form lifecycle management.